

Architectural Optimization of Large Language Model Tokenization for Traditional Chinese in Agentic Software Engineering Environments

The Fundamental Asymmetry of Tokenization in Neural Architectures

The operational deployment of Large Language Models (LLMs) in software engineering workflows has precipitated a profound shift in developer productivity, enabling natural language to drive complex code generation, debugging, and repository orchestration. However, the efficacy of this paradigm is intrinsically bound to the underlying computational mechanism of tokenization. Tokenization, the process by which human-readable text is compressed into discrete mathematical units of meaning, is not a linguistically neutral protocol. In mixed code-and-language environments—where programming syntaxes are overwhelmingly derived from Latin scripts and developer intent is articulated in non-English logographic systems—the mechanical translation of strings into tokens generates severe computational friction.¹

For developers communicating in Traditional Chinese (TC), this friction manifests as an exponential amplification in token consumption. This bloat directly inflates operational expenditures and artificially constrains the model's effective context window, precipitating premature context loss during extended agentic loops.¹ Understanding the etiology of this disparity requires a granular examination of how subword tokenization algorithms compress linguistic data.

The Mechanics of Subword Fragmentation and Logographic Penalties

Modern foundation models predominantly utilize data-driven subword tokenization algorithms, most notably Byte-Pair Encoding (BPE), WordPiece, or Unigram-based SentencePiece frameworks. These algorithms initialize with a foundational vocabulary of bytes or characters and iteratively merge adjacent pairs based strictly on their statistical frequency within the training corpus.¹ Because the vast majority of corpora used to train state-of-the-art models are predominantly English-centric, the resulting token vocabularies are heavily optimized for English lexicon, common Latin affixes, and standard programming syntaxes.¹

In English, explicit word boundaries (spaces) and highly repetitive sub-structures allow BPE algorithms to optimally compress phrases. For instance, the phrase "machine learning" maps flawlessly to two tokens in English-optimized vocabularies.¹ Conversely, logographic scripts

such as Chinese eschew natural word segmentation. A phrase like "机器学习" is inherently treated by a naïve tokenizer as a continuous stream of characters.¹ While each individual Chinese character possesses extraordinary semantic density—often encapsulating concepts that require multiple English words to articulate—the tokenization algorithm fails to recognize this density unless the specific character combination has been explicitly prioritized in the model's limited CJK (Chinese, Japanese, Korean) vocabulary space.¹

This architectural bias becomes acutely problematic when comparing Simplified Chinese (SC) and Traditional Chinese (TC). Because Simplified Chinese constitutes a significantly larger proportion of the scraped internet data used in pre-training, BPE algorithms are more likely to assign dedicated, single-token representations to common SC characters and multi-character phrases.⁴ Traditional Chinese characters, which are structurally more complex and statistically less prevalent in Western-curated datasets, frequently trigger the tokenizer's fallback mechanisms.⁴ Rather than mapping a TC character to a single semantic token, the tokenizer fragments the character into multiple subword units or, in worst-case scenarios, raw UTF-8 bytes.⁴ Consequently, conveying the exact same semantic instruction requires markedly more tokens in Traditional Chinese than in Simplified Chinese, and frequently two to three times more tokens than in English.¹

Cognitive Degradation and Economic Implications

The subword fragmentation of Traditional Chinese characters precipitates two critical systemic failures within the context of coding agents. The primary constraint is economic. Commercial LLM Application Programming Interfaces (APIs) bill explicitly by token volume. With output tokens generally priced three to five times higher than input tokens, the financial penalty for a developer utilizing an agentic loop in Traditional Chinese is compounding and severe.¹

The secondary, and perhaps more insidious, constraint is cognitive degradation. The self-attention mechanism, which governs how transformer architectures comprehend context, scales quadratically with sequence length.⁶ When a semantic concept is fragmented across multiple arbitrary subword tokens, the model is forced to expend a disproportionate amount of its computational attention budget re-assembling the lexical integrity of the word before it can deduce the broader syntactic instruction.⁶ This dual processing burden dilutes the model's reasoning capabilities. Research indicates that models compelled to reason over highly fragmented token streams exhibit elevated rates of hallucination, suffer rapid context degradation in deep codebases, and demonstrate reduced coherence in multi-turn planning.⁶ Empirical studies evaluating the premise of "vibe coding" in Chinese have demonstrated that despite the character-level density of the language, the tokenizer-induced sequence bloat actively lowers the task resolution rate and success probability on software engineering benchmarks like SWE-bench Lite.³

Character Efficiency Versus Token Efficiency

The narrative surrounding token efficiency is often conflated with character density. Token

counting experiments utilizing modern tokenizers reveal that Chinese text maps to substantially fewer characters per token—approximately 1.33 characters per token—compared to English, which averages 4.75 characters per token.³ However, this "character efficiency" does not translate to "token efficiency." The fact that Chinese characters encode more semantic information per glyph is an intrinsic property of the writing system; the fragmentation of those glyphs into multiple subword units is a design choice inherent to tokenizer training.³ Controlled experiments measuring total token consumption confirm that there is no intrinsic computational advantage to logographic languages when processed through English-dominant BPE vocabularies.³

To comprehend the full scope of this issue and engineer middleware interventions, it is imperative to dissect the tokenization behavior and architectural mechanics of the predominant coding agent platforms: Anthropic's Claude Code, Google's Gemini CLI, and OpenAI's Codex CLI.

Tokenizer Profiles and Cost Mechanics Across Agent Platforms

The disparity in multilingual tokenization efficiency is not uniform; it varies drastically depending on the specific model architecture, tokenizer iteration, and the underlying pricing schema.

Anthropic Claude Architecture

Anthropic's Claude models exhibit a complex and evolving tokenization landscape. Early iterations of the Claude tokenizer demonstrated relatively robust and balanced multilingual support compared to contemporaneous GPT architectures, achieving a marginally more efficient mapping for CJK languages.¹ However, sequential updates to the tokenizer have prioritized code execution fidelity and English prose generation over absolute multilingual compression.⁹

The transition from the Claude 4.6 tokenizer to the Opus 4.7 tokenizer illustrates a deliberate shift in this optimization strategy. Empirical analysis of the Opus 4.7 tokenizer reveals that natural English content and code structures map to 1.20–1.47x more tokens than under the 4.6 architecture.⁹ This token inflation is the direct consequence of utilizing shorter, more granular sub-word merges to prevent hallucination during complex code generation.⁹ Crucially, CJK content only shifted by 1.005–1.07x during this upgrade, indicating that the new vocabulary distribution did not meaningfully expand its native support for Chinese logographs.⁹

Claude 3.5 Sonnet, a model heavily utilized in the Claude Code CLI due to its high inference speed and formidable reasoning capabilities, illustrates the ongoing economic friction. Priced at \$3.00 per million input tokens and \$15.00 per million output tokens, the model provides an expansive 200K token context window.⁵ While benchmarking indicates that Claude 3.5 Sonnet consumes approximately 34% fewer tokens than GPT-4o on specific multilingual summarization tasks involving Traditional Chinese, the baseline inefficiency remains a

bottleneck for developers.¹³ In agentic workflows, where the model recursively reads files, executes bash commands, and analyzes lengthy stack traces, Traditional Chinese developers find their context windows filling disproportionately faster than their English-speaking counterparts.¹²

Google Gemini SentencePiece Optimization

Google's Gemini architectures utilize a Unigram-based SentencePiece tokenizer, distinguishing it from standard BPE by utilizing a probabilistic model to determine the optimal subword sequence from a vast vocabulary pool.¹⁵ The Gemini 3 Pro architecture introduced targeted, algorithmic optimizations specifically designed to enhance Chinese tokenization efficiency.¹⁶

These enhancements included a substantial increase in the proportion of Chinese data within the tokenizer's training corpus and the introduction of a dynamic, language-aware fallback mechanism.¹⁶ This allows high-frequency Chinese character combinations to receive logical, single-token representations. When encountering out-of-vocabulary terms, the tokenizer prioritizes representing them using known semantic character roots rather than degrading immediately to inefficient, single-character or byte-level fragmentation.¹⁶ Analytical testing demonstrates that this mixed-granularity tokenization reduces token consumption for Chinese technical documents by approximately 24% compared to previous Gemini iterations.¹⁶

Despite these structural improvements, Gemini's universal tokenization of Traditional Chinese remains demonstrably less efficient than architectures specifically trained for CJK dominance. Comparative studies against natively Chinese models, such as DeepSeek V3.2, reveal that for identical multilingual translation outputs, Gemini consumes 2.0 to 2.56 times more completion tokens.¹⁵ DeepSeek's output efficiency registers at 2.02–4.16 characters per token, whereas Gemini registers at a fragmented 0.82–1.84 characters per token for the same text.¹⁵ This underscores the limitation of a universal vocabulary when confronted with the vast morphological complexity of Traditional Chinese.¹⁵

OpenAI Codex and the cl100k_base Architecture

OpenAI's models, including those utilized within the Codex CLI, operate on the cl100k_base tokenizer. This vocabulary is aggressively optimized for English prose and popular programming syntaxes. Character-to-token ratio studies consistently demonstrate that this tokenizer penalizes Chinese text, requiring approximately 15% more tokens per equivalent semantic content compared to tokenizers with a more balanced linguistic distribution.³

Within the interactive terminal environment of the Codex CLI, this inefficiency is vastly magnified.¹⁷ Agentic loops inherently ingest massive volumes of shell command outputs, compilation errors, stack traces, and local test results.¹⁸ Because Codex is frequently utilized for iterative debugging, the amalgamation of verbose, English-heavy tracebacks intermingled with Traditional Chinese developer queries creates severe sequence bloat, exhausting the token budget well before a complex task is resolved.¹⁸

Comparative Token Efficiency and Cost Matrix

To contextualize the operational landscape for a Traditional Chinese developer, the following table summarizes the tokenization mechanics and resultant penalties across major foundation models:

Model Architecture	Tokenizer Framework	CJK Tokenization Strategy	TC to English Token Overhead	Context Window & Cost Mechanics
Claude (Anthropic)	BPE (Dynamic Iterations)	Moderate merging; prioritizing granular merges for precise code execution.	High (2x - 3x penalty)	Aggressive auto-compaction; 1M token limits available but costly at scale.
Gemini (Google)	SentencePiece (Unigram)	Language-aware fallback; mixed-granularity root matching in v3.	Moderate (1.5x - 2x penalty)	Massive 1M+ context window; highly granular CLI hook system for intervention.
Codex (OpenAI)	cl100k_base (BPE)	English/Code dominant; heavy subword fragmentation for CJK characters.	Very High (>3x penalty)	Susceptible to terminal output bloat; relies heavily on community proxy tools.
Qwen / GLM	Native Chinese BPE	Expanded CJK vocabulary; native word-level mapping for	Inverted (TC/SC is more efficient than English)	Highly cost-efficient, though may trail frontier models in advanced English

		logographs.		code logic.
--	--	-------------	--	-------------

Architectural Mechanics of CLI Coding Agents

To engineer a robust middleware solution that mitigates the token penalty without forcing the developer to abandon frontier reasoning models like Claude 3.5 Sonnet, intervention must occur at the Command Line Interface (CLI) level. Modern CLI coding agents feature event-driven lifecycle hooks that allow external scripts to intercept, validate, modify, or block data as it flows between the user, the local filesystem, and the remote LLM API.²¹ Understanding the precise asymmetry of these hook systems is critical for implementing a successful token-reduction stack.

Claude Code Event Lifecycle and Architectural Asymmetry

Anthropic's Claude Code orchestrates an autonomous agentic loop governed by configuration files (e.g., global `~/.claude/settings.json` or project-local `.claude/settings.json`).²⁴ The execution pipeline surfaces discrete events, enabling developers to bind shell commands or Node.js scripts to specific phases of the agent's cognitive loop.²²

The standard Claude Code lifecycle includes the following highly specific hooks:

- SessionStart:** Triggered immediately upon initialization or resumption of a session. It is primarily utilized to load environmental context, initialize telemetry, or enforce guardrails before the agent assumes control.²³
- UserPromptSubmit:** Triggered the moment a developer inputs a conversational query. The hook receives the raw prompt text via a standard input JSON payload.²⁸
- PreToolUse:** Fired explicitly before Claude executes any system tool (e.g., executing a bash command, reading a file, or attempting a git commit).²² This hook is highly privileged; it allows developers to validate input, block dangerous operations entirely, or critically, modify tool arguments dynamically via the `updatedInput` JSON field.²²
- PostToolUse:** Fired immediately after a tool executes but prior to the system relaying the output back to Claude.²⁴ It is optimal for intercepting raw terminal outputs, running linters, or compressing verbose log files.²⁵
- PreCompact:** Fired immediately before Claude's internal auto-compaction algorithm truncates the conversation history to salvage token space. This is used to persist critical state variables before they are erased from the context window.¹⁴

The Fundamental Asymmetry in Claude Code Hooks: While the Claude Code architecture is robust, it harbors a critical limitation for token optimization middleware. The `PreToolUse` hook natively supports the `updatedInput` parameter, allowing a hook to seamlessly alter the parameters of a tool before execution.²⁹ However, the `UserPromptSubmit` hook currently lacks

an analogous updatedPrompt capability.²⁹

If a user submits a prompt in Traditional Chinese, a UserPromptSubmit hook can read the prompt, block it entirely (by returning an exit code of 2), or append supplementary text via the additionalContext field.²⁸ It *cannot* seamlessly rewrite, compress, or replace the user's text before it is embedded into the LLM's context window.¹⁴ This missing updatedPrompt feature breaks the symmetry of the interceptor pattern and precludes the ability to implement an invisible, native translation layer directly within the .claude/settings.json hook system.²⁹

Gemini CLI Granularity and the BeforeModel Hook

In stark contrast, the Gemini CLI offers an exceptionally granular event framework that specifically accommodates prompt mutation and API-level interception.²¹ The lifecycle hooks in Gemini mirror the JSON-over-stdin contract utilized by Claude Code but introduce events explicitly designed for advanced prompt engineering.²³

The most critical event in the Gemini lifecycle is the BeforeModel hook.²³ This hook triggers after the agent planning phase has concluded but immediately prior to the execution of the actual LLM API HTTP request.²³ A script attached to the BeforeModel event is granted full authority to modify the prompt payload, swap the target model dynamically, or inject mocked responses directly into the stream.²³

For a Traditional Chinese developer using Gemini, a BeforeModel hook can intercept the TC input, pass it to a local translation daemon, and seamlessly return the English equivalent to the CLI.³⁴ The Gemini agent subsequently processes the highly compressed English translation, effectively bypassing the SentencePiece token penalty without any disruption to the developer's native linguistic workflow.³⁴

Codex CLI Proxy Ecosystems

The Codex CLI ecosystem, while powerful, provides less robust, natively documented lifecycle hooks for pre-processing prompts compared to Gemini.³⁶ Consequently, the community has pivoted toward interceptor and proxy paradigms. Tools such as the Rust Token Killer (RTK) function as intermediary proxy layers, intercepting the standard output generated by shell tools and aggressively stripping ambient noise before it reaches the Codex context manager.³⁶ This architectural workaround routinely saves 60% to 90% of the tokens generated by background operations, validating the necessity of local middleware for token optimization.¹⁸

Strategic Vectors for Token Optimization Middleware

Given the specific architectural constraints of these agent platforms, resolving the Traditional Chinese token penalty requires a multi-vector middleware strategy. This strategy combines real-time translation, prompt architecture manipulation, codebase context distillation, and selective Model Context Protocol (MCP) data retrieval.

Vector 1: Real-Time Script Translation (TC to SC/English)

The most mathematically direct method to circumvent the token penalty is pre-translation. While translating Traditional Chinese to Simplified Chinese provides a nominal token saving due to the SC-heavy training bias inherent in BPE tokenizers⁴, translating TC directly into English yields the absolute maximum compression ratio.¹

To implement this without introducing severe API latency into the interactive developer workflow, the translation must be executed entirely locally.

The OpenCC Architecture and Runtime Environments: OpenCC (Open Chinese Convert) is a high-performance, open-source translation engine explicitly designed to facilitate conversions between Traditional and Simplified Chinese, alongside regional idiomatic variants.³⁹ The engine utilizes a Forward Maximum Matching (FMM) algorithm operating against highly optimized, pre-compiled dictionary tries (the .ocd2 format), ensuring that translations occur in single-digit milliseconds.³⁹

For CLI hook integration, the choice of runtime environment is paramount. Executing an external Python script via a hook can introduce latency due to interpreter startup overhead and the lack of true concurrency under load.⁴¹ Utilizing node-opencc (a pure JavaScript implementation) or the Rust binding (opencc-fmmseg) is highly preferable.⁴⁰ Node.js, leveraging its event-driven, non-blocking I/O model, demonstrates vastly superior throughput and lower latency for concurrent hook executions compared to Python, rendering it the optimal runtime for agentic middleware that may fire dozens of times per minute during an active session.⁴¹

While OpenCC strictly handles orthographic Chinese script variants, leveraging a small, heavily quantized local LLM (such as a localized Ollama instance running Qwen2.5) can facilitate rapid, context-aware TC-to-English translation.⁴⁵ A local translation middleware intercepts the input, resolves programming pronouns accurately based on the terminal context, isolates and protects markdown syntax and code blocks, and feeds the pure English result to the primary cloud agent.⁴⁵

Vector 2: Prompt Compression and the "Caveman Mode" Paradigm

Beyond translation, overall conversational verbosity consumes substantial tokens and paradoxically increases the probability of model hallucination. An emerging methodology to combat this inefficiency is "Caveman Mode"—the strict enforcement of ultra-concise, telegraphic interaction protocols via prompt engineering.⁷

By injecting a rigid system prompt that overrides the foundation model's default conversational verbosity, developers can instruct the LLM to output only raw code diffs, omit all explanatory pleasantries, and summarize logical transitions using minimalist syntax.⁴⁶ Empirical testing on LLMs validates that restricting large models to absolute brevity improves logic accuracy by preventing "overelaboration"—a well-documented phenomenon wherein highly verbose

models explore irrelevant tangents, inject useless hedges, and ultimately lose track of the core computational objective.⁴⁶ A 2026 paper (arXiv:2604.00025) demonstrated that imposing strict brevity constraints improved LLM accuracy by 26 percentage points on problems where verbosity traditionally caused logical drift.⁴⁶

For a Traditional Chinese developer, configuring the agent to "think in Chinese" internally but forcing it to summarize operations in a "caveman" English schema can halve context window consumption.⁷ In Claude Code, this is implemented by deploying a foundational SKILL.md file within the .claude/skills directory, or by utilizing a PostToolUse hook that parses the model's intermediate outputs and strictly enforces brevity prior to rendering the text in the local terminal.⁴⁶

Vector 3: Codebase Context Distillation and Shadow Files

Coding agents exhaust context windows exponentially by executing full-file read operations when only a discrete function signature is required. This problem is drastically exacerbated if the target file contains dense Traditional Chinese comments, documentation, or string literals, as the token count balloons immediately upon ingestion.

Selective Comment Translation and the Shadow File Architecture: While stripping all comments from a file saves tokens, it deprives the LLM of semantic context critical for accurate code translation and structural reasoning.⁵⁰ A superior middleware approach involves the orchestration of shadow files.

When Claude invokes the Read tool, a bash script or Node.js module attached to the PreToolUse hook intercepts the request.²⁹ This script generates a temporary "shadow copy" of the target file.²⁹ Within this shadow copy, the middleware utilizes Abstract Syntax Tree (AST) parsing to cleanly separate code execution logic from comments. The programming syntax remains untouched, while all TC comments are passed through a local OpenCC instance (translating to SC) or a local translation LLM (translating to English) before being saved.

The hook then utilizes the updatedInput JSON field to transparently redirect Claude's read operation away from the original file and toward the token-optimized shadow copy.²⁹ When Claude attempts to utilize the Write or Edit tools, it modifies the shadow file. A PostToolUse hook immediately captures this event, executing a reverse-translation or applying an AST-based diff, thereby restoring the original Traditional Chinese comments before committing the final changes to the primary repository.²⁹

Trace Compaction: Terminal outputs, specifically stack traces and automated test logs, are notorious token sinks within agentic loops.¹⁸ A local trace compactor hook acts as middleware intercepting shell command outputs (triggering on the PostToolUse hook associated with the Bash tool). The middleware parses the highly verbose traceback, extracts only the critical exception type, the relevant file execution frames, and the failing code lines, and formats this data into a highly dense XML tag (e.g., <COMPACT_TRACEBACK>).⁵² This process effortlessly reduces a standard 250-token error log down to approximately 40 tokens while retaining 100%

of the diagnostic signal required by the LLM.⁵²

Vector 4: Model Context Protocol (MCP) Integration

The Model Context Protocol (MCP) represents a standardized, open-source architectural paradigm allowing AI agents to interact securely with external datasets and APIs without forcing entire schema definitions into the limited context window.⁵³ Rather than an agent executing a brute-force file read on a 3000-line script to understand a function, an MCP server processes the codebase externally, constructing a metadata index of dependency graphs, class definitions, and function signatures.⁵⁴

When the LLM requires an understanding of system architecture, it queries the MCP server for highly specific metadata.⁵⁶ Implementing a custom MCP server, such as `mcp-codebase-index`, enables the agent to execute surgical, structured lookups.⁵⁶ In rigorous benchmark testing against a production repository, dependency lookups executed via MCP achieved a staggering 99% token reduction compared to native file reading, dropping the accumulated context load from ~32,000 tokens to a mere ~900 tokens over a ten-turn session.⁵⁶

For Traditional Chinese developers, isolating the agent from the raw, TC-heavy codebase files and restricting it to ingesting AST-derived metadata via MCP bypasses the subword tokenization penalty almost entirely for read and structural operations.⁵⁶ By utilizing the command `claude mcp add` to connect a localized, English-translating MCP indexer, developers can maintain massive TC codebases while operating the agent entirely within an optimized English context.⁵⁵

Architectural Blueprint: The Claude Code Interceptor Stack

Because Claude Code is the primary platform target, the proposed middleware solution must elegantly sidestep the architectural absence of the `updatedPrompt` capability in the `UserPromptSubmit` hook.²⁹ Relying strictly on the native `.claude/settings.json` is insufficient for comprehensive bidirectional translation; therefore, a wrapper-based CLI interceptor architecture is required.

To actualize this token optimization strategy, the following cohesive middleware stack is recommended:

1. The CLI Wrapper Layer (Prompt Interception)

Instead of invoking the `claude` binary directly, the developer invokes a Node.js or Python-based CLI interceptor (functionally similar to open-source wrappers like `Claude-Code-Interceptor` or `claudish`).⁵⁹

- **Mechanism:** The interceptor captures the user's raw terminal input before it ever reaches the Anthropic SDK. If the input contains Traditional Chinese, the wrapper routes it

through a localized translation engine. Leveraging node-openccc provides zero-I/O TC-to-SC translation, while routing through a local Ollama instance provides high-fidelity TC-to-English translation.⁴³

- **Execution:** The translated string is then dynamically passed as standard input to the underlying, hidden claude process. This architectural bypass guarantees that Claude's context window only ever receives highly optimized, token-efficient text, circumventing the limitations of the native hook system entirely.²⁹

2. Native File Masking via PreToolUse

When Claude attempts to read the local filesystem, the native CLI hooks must intercept the payloads.

- **The Implementation:** Configure the project .claude/settings.json to utilize PreToolUse and PostToolUse scripts. When the Read tool is invoked, the hook creates a shadow copy of the target file.²⁹ AST parsers strip or translate the TC comments. The hook subsequently returns a JSON payload containing the updatedInput property, seamlessly redirecting Claude to read the sanitized shadow file instead of the token-heavy original.²⁹

3. Output Filtration and the StatusLine

Because CLI output streams continuously back to the developer's terminal, verbose agent logic must be constrained systemically.

- **The Implementation:** A SKILL.md file must be deployed in the .claude/skills directory, utilizing the Agent Skills open standard to enforce the "Caveman Mode" directive.⁴⁶ Furthermore, a script attached to the PreCompact hook must actively monitor the context_window.remaining_percentage variable exposed by the StatusLine tool.¹⁴ This allows the middleware to actively manage state backups and force context distillation before Claude performs its lossy, default auto-compaction routine.¹⁴

Performance Modeling and Cost-Benefit Synthesis

The integration of this multi-layered interceptor stack fundamentally alters the operational economics of agentic software engineering for non-English users.

Middleware Component	Core Optimization Mechanism	Estimated Token Savings	Local Latency Impact
CLI Interceptor Wrapper	TC to EN/SC translation of user prompts prior to	50% - 70% per prompt	~100ms (Node-OpenCC) to

	claude binary execution.		~800ms (Local LLM)
Shadow File Masking	updatedInput redirection to AST-parsed files with stripped/translated TC comments.	60% - 80% per file read	~50ms (Regex/AST parsing)
Trace Compactor Hook	PostToolUse interception of Bash shell outputs; XML-tagged exception extraction.	85% - 95% per shell error	~20ms (Regex/Text processing)
AST-Driven MCP Indexer	Replaces brute-force file reads with structured dependency metadata lookups.	80% - 95% per multi-file query	Zero added API latency; local compute only
Caveman Mode SKILL.md	System prompt overriding verbosity; enforcing structural brevity in API outputs.	40% - 75% reduction in output tokens	Improves generation speed (fewer tokens to predict)

By systematically intercepting, translating, and distilling inputs and outputs, this architecture artificially imposes the high computational compression ratios of the English language onto Traditional Chinese workflows. While the introduction of local processing incurs a minor initial setup overhead and fractional latency during script execution, the resultant dividends are overwhelmingly positive. Token consumption per engineering task drops precipitously, operational API costs are minimized, and the effective capacity of the model's context window is vastly expanded without requiring costly upgrades to 1M token tiers.⁵

More critically, by insulating the foundation model from the cognitive scatter introduced by subword fragmentation, the agent's reasoning capability, logic retention, and code generation accuracy are strictly maintained at frontier levels.⁸ This multi-layered middleware architecture ensures that Traditional Chinese developers can leverage state-of-the-art AI tooling with the exact same computational efficiency, precision, and financial viability as their English-speaking counterparts.

引用的著作

1. Tokenization Explained: Why Chinese Costs More? | Lafu Code, 檢索日期: 4月 30, 2026, <https://lafucode.com/en/posts/token-principle-chinese-vs-english-cost>
2. To Merge or Not to Merge: The Pitfalls of Chinese Tokenization in General-Purpose LLMs, 檢索日期: 4月 30, 2026, <https://digitalorientalist.com/2025/02/04/to-merge-or-not-to-merge-the-pitfalls-of-chinese-tokenization-in-general-purpose-llms/>
3. Chinese Language Is Not More Efficient Than English in Vibe Coding: A Preliminary Study on Token Cost and Problem-Solving Rate - arXiv, 檢索日期: 4月 30, 2026, <https://arxiv.org/html/2604.14210v1>
4. A Perspective on Traditional Chinese LLM | by Audrey | Medium, 檢索日期: 4月 30, 2026, <https://medium.com/@colaflyfly/a-perspective-on-traditional-chinese-llm-ba55b1eb2e92>
5. What Is Token-Based Pricing for AI Models - MindStudio, 檢索日期: 4月 30, 2026, <https://www.mindstudio.ai/blog/token-based-pricing>
6. Why Non-English Speakers Pay More for AI | by Craig Trim | Medium, 檢索日期: 4月 30, 2026, <https://medium.com/@craigtrim/why-non-english-speakers-pay-more-for-ai-eb6db7d5b67c>
7. I made 10 agents think in Chinese to save token. Here is why. : r/ClaudeCode - Reddit, 檢索日期: 4月 30, 2026, https://www.reddit.com/r/ClaudeCode/comments/1sntyg3/i_made_10_agents_think_in_chinese_to_save_token/
8. Chinese Language Is Not More Efficient Than English in Vibe Coding - arXiv, 檢索日期: 4月 30, 2026, <https://arxiv.org/pdf/2604.14210>
9. I Measured Claude 4.7's New Tokenizer. Here's What It Costs You., 檢索日期: 4月 30, 2026, <https://www.claudecodecamp.com/p/i-measured-claude-4-7-s-new-tokenizer-here-s-what-it-costs-you>
10. Claude's tokenizers have actually been getting less efficient over the years (I ... - Hacker News, 檢索日期: 4月 30, 2026, <https://news.ycombinator.com/item?id=47807951>
11. Claude Token Counter, now with model comparisons - Simon Willison's Weblog, 檢索日期: 4月 30, 2026, <https://simonwillison.net/2026/apr/20/claude-token-counts/>
12. Introducing Claude 3.5 Sonnet - Anthropic, 檢索日期: 4月 30, 2026,

- <https://www.anthropic.com/news/claude-3-5-sonnet>
13. LLM Token Efficiency Cost Benchmarking: APAC Data - Branch8, 檢索日期: 4月 30, 2026, <https://branch8.com/posts/llm-token-efficiency-cost-benchmarking-apac-works>
 14. Claude Code Context Buffer: The 33K-45K Token Problem, 檢索日期: 4月 30, 2026, <https://claudefa.st/blog/guide/mechanics/context-buffer-management>
 15. Same Article Translation Token Difference 2.5x: Gemini vs DeepSeek Tokenizer Efficiency Actual Test Comparison - Apiyi.com Blog, 檢索日期: 4月 30, 2026, <https://help.apiyi.com/en/gemini-vs-deepseek-tokenizer-efficiency-same-text-different-token-cost-guide-en.html>
 16. Gemini 3 Pro's Chinese Tokenization Efficiency Improves by 24%, Delivering Significant Cost Benefits for Office Scenarios - BigGo Finance, 檢索日期: 4月 30, 2026, <https://finance.biggo.com/news/2PLcrZ0BvbifYyetITFg>
 17. Codex CLI - OpenAI Developers, 檢索日期: 4月 30, 2026, <https://developers.openai.com/codex/cli>
 18. Quick Hack: Save up to 99% tokens in Codex - Reddit, 檢索日期: 4月 30, 2026, https://www.reddit.com/r/codex/comments/1rmo4oj/quick_hack_save_up_to_99_tokens_in_codex/
 19. Features - Codex CLI | OpenAI Developers, 檢索日期: 4月 30, 2026, <https://developers.openai.com/codex/cli/features>
 20. Subscription/API Comparison Table by Token Cost?: r/opencodeCLI - Reddit, 檢索日期: 4月 30, 2026, https://www.reddit.com/r/opencodeCLI/comments/1r5p9uy/subscriptionapi_comparison_table_by_token_cost/
 21. Tailor Gemini CLI to your workflow with hooks - Google Developers Blog, 檢索日期: 4月 30, 2026, <https://developers.googleblog.com/tailor-gemini-cli-to-your-workflow-with-hooks/>
 22. Hooks reference - Claude Code Docs, 檢索日期: 4月 30, 2026, <https://code.claude.com/docs/en/hooks>
 23. Gemini CLI hooks, 檢索日期: 4月 30, 2026, <https://geminicli.com/docs/hooks/>
 24. claude-code:hook | Skills Marketplace - LobeHub, 檢索日期: 4月 30, 2026, <https://lobehub.com/de/skills/bendrucker-claude-hook>
 25. Claude Code Hooks: A Practical Guide to Workflow Automation - DataCamp, 檢索日期: 4月 30, 2026, <https://www.datacamp.com/tutorial/claude-code-hooks>
 26. F3: Foundation-First Framework | MethodFactory, 檢索日期: 4月 30, 2026, <https://methodfactory.com/frameworks/f3-framework.html>
 27. claude-agent-sdk • claude-skills • jezweb • Skills • Registry - Tessl, 檢索日期: 4月 30, 2026, <https://tessl.io/registry/skills/github/jezweb/claude-skills/claude-agent-sdk>
 28. Automate workflows with hooks - Claude Code Docs, 檢索日期: 4月 30, 2026, <https://code.claude.com/docs/en/hooks-guide>
 29. Feature Request: Add `updatedPrompt` support to `UserPromptSubmit` hook · Issue #27365 · anthropics/claude-code - GitHub, 檢索日期: 4月 30, 2026,

- <https://github.com/anthropics/claude-code/issues/27365>
30. The Claude Developer Guide Agent SDK Reference — Python SDK - GoPenAI, 檢
索日期: 4月 30, 2026,
[https://blog.gopenai.com/the-claude-developer-guide-agent-sdk-reference-pyt
hon-sdk-921cb76dc7f5](https://blog.gopenai.com/the-claude-developer-guide-agent-sdk-reference-python-sdk-921cb76dc7f5)
 31. HOW TO USE `HOOKS` on CLAUDE CODE CLI: Intelligent Git workflow
automation for Claude Code that creates checkpoint commits on every file
change and squashes them into meaningful task commits. : r/ClaudeAI - Reddit,
檢索日期: 4月 30, 2026,
[https://www.reddit.com/r/ClaudeAI/comments/1m083kb/how_to_use_hooks_on_c
laude_code_cli_intelligent/](https://www.reddit.com/r/ClaudeAI/comments/1m083kb/how_to_use_hooks_on_claude_code_cli_intelligent/)
 32. Feature request: Allow UserPromptSubmit hooks to modify prompt text · Issue
#34390 · anthropics/claude-code - GitHub, 檢索日期: 4月 30, 2026,
<https://github.com/anthropics/claude-code/issues/34390>
 33. Feature: Comprehensive Hooking System · Issue #9070 ·
google-gemini/gemini-cli - GitHub, 檢索日期: 4月 30, 2026,
<https://github.com/google-gemini/gemini-cli/issues/9070>
 34. Gemini CLI configuration, 檢索日期: 4月 30, 2026,
<https://geminicli.com/docs/reference/configuration/>
 35. [FEATURE] Synthetic Response Injection for Testing and Caching · Issue #21536 ·
anthropics/claude-code - GitHub, 檢索日期: 4月 30, 2026,
<https://github.com/anthropics/claude-code/issues/21536>
 36. Codex CLI now support 3 hooks (Claude code has 23) - Reddit, 檢索日期: 4月 30,
2026,
[https://www.reddit.com/r/codex/comments/1rykz2w/codex_cli_now_support_3_ho
oks_claude_code_has_23/](https://www.reddit.com/r/codex/comments/1rykz2w/codex_cli_now_support_3_hooks_claude_code_has_23/)
 37. Hook would be a great feature · openai codex · Discussion #2150 - GitHub, 檢索日
期: 4月 30, 2026, <https://github.com/openai/codex/discussions/2150>
 38. Add RTK Directly Into Codex CLI to Reduce Token Usage 60–90% by Filtering
Shell Command Output #19001 - GitHub, 檢索日期: 4月 30, 2026,
<https://github.com/openai/codex/issues/19001>
 39. OpenCC/AGENTS.md at master - GitHub, 檢索日期: 4月 30, 2026,
<https://github.com/BYVoid/OpenCC/blob/master/AGENTS.md>
 40. opencc.rs - source, 檢索日期: 4月 30, 2026,
https://docs.rs/opencc-fmmseg/latest/src/opencc_fmmseg/opencc.rs.html
 41. Comparing Node.js and Python Performance with the Official OpenAI Client -
API, 檢索日期: 4月 30, 2026,
[https://community.openai.com/t/comparing-node-js-and-python-performance-w
ith-the-official-openai-client/787874](https://community.openai.com/t/comparing-node-js-and-python-performance-with-the-official-openai-client/787874)
 42. Pure C OpenCL vs Python OpenCL performance - Stack Overflow, 檢索日期: 4月
30, 2026,
[https://stackoverflow.com/questions/23372734/pure-c-opencl-vs-python-opencl
-performance](https://stackoverflow.com/questions/23372734/pure-c-opencl-vs-python-opencl-performance)
 43. compulim/node-opencc: OpenCC implementation for pure Node.js - GitHub, 檢
索日期: 4月 30, 2026, <https://github.com/compulim/node-opencc>

44. What are the biggest differences between Node and Python for web? - Reddit, 檢
索日期: 4月 30, 2026,
https://www.reddit.com/r/node/comments/1f70px3/what_are_the_biggest_differences_between_node_and/
45. kimi230/claude-ts - GitHub, 檢索日期: 4月 30, 2026,
<https://github.com/kimi230/claude-ts>
46. Claude Code Caveman Mode: the Skill That Cuts Tokens Without Breaking Code,
檢索日期: 4月 30, 2026,
<https://pasqualepillitteri.it/en/news/846/claude-code-caveman-mode-token-saving>
47. Taught Claude to talk like a caveman to use 75% less tokens. : r/ClaudeAI - Reddit,
檢索日期: 4月 30, 2026,
https://www.reddit.com/r/ClaudeAI/comments/1sble09/taught_claude_to_talk_like_a_caveman_to_use_75/
48. Extend Claude with skills - Claude Code Docs, 檢索日期: 4月 30, 2026,
<https://code.claude.com/docs/en/skills>
49. Create and use skills in claude code CLI - YouTube, 檢索日期: 4月 30, 2026,
<https://www.youtube.com/watch?v=M-rJrFyP1s>
50. Preserve code comments · Issue #795 · rtk-ai/rtk - GitHub, 檢索日期: 4月 30, 2026
, <https://github.com/rtk-ai/rtk/issues/795>
51. Revisiting the Role of Natural Language Code Comments in Code Translation -
arXiv, 檢索日期: 4月 30, 2026, <https://arxiv.org/html/2601.16661v1>
52. tarekziade/claude-tools: Hooks and tools for Claude Code - GitHub, 檢索日期: 4
月 30, 2026, <https://github.com/tarekziade/claude-tools>
53. Code execution with MCP: building more efficient AI agents - Anthropic, 檢索日期
: 4月 30, 2026,
<https://www.anthropic.com/engineering/code-execution-with-mcp>
54. An Introduction to MCP and Authorization | Auth0, 檢索日期: 4月 30, 2026,
<https://auth0.com/blog/an-introduction-to-mcp-and-authorization/>
55. Introducing advanced tool use on the Claude Developer Platform - Anthropic, 檢
索日期: 4月 30, 2026,
<https://www.anthropic.com/engineering/advanced-tool-use>
56. I built an MCP to significantly reduce your token consumption : r/claude - Reddit,
檢索日期: 4月 30, 2026,
https://www.reddit.com/r/claude/comments/1r71lyb/i_built_an_mcp_to_significantly_reduce_your_token/
57. Build an MCP Server in Minutes: Practical Guide to LLM Integration, 檢索日期: 4月
30, 2026, <https://www.gravitee.io/blog/mcp-server-explained>
58. Connect Claude Code to tools via MCP, 檢索日期: 4月 30, 2026,
<https://code.claude.com/docs/en/mcp>
59. eleqtrizit/Claude-Code-Interceptor - GitHub, 檢索日期: 4月 30, 2026,
<https://github.com/eleqtrizit/Claude-Code-Interceptor>
60. GitHub - MadAppGang/clauidish: Claude Code. Any Model. The most powerful AI
coding agent now speaks every language., 檢索日期: 4月 30, 2026,
<https://github.com/MadAppGang/clauidish>

61. Compared actual usage costs for Chinese AI models. Token efficiency changes everything., 檢索日期: 4月 30, 2026,
https://www.reddit.com/r/LocalLLaMA/comments/1p8ven0/compared_actual_usage_costs_for_chinese_ai_models/